Table of Contents

Choosing React Built Tool **2**

ASP.NET Core React Template 2
Vite + .NET Core 3
Next.js 3
React Native 4
Create React App (CRA) 4

Creating the React helloworld project **5**

Explanation of files **8**

Demo: React HelloWorld

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT/Gemini.

Choosing React Built Tool

We've chosen **Vite** as our front-end build tool for React in this course, coupled with an ASP.NET Core API Service as the backend. This setup is a very popular and professional practice in the industry, as it provides a clear separation of the front-end and back-end.

According to the State of JavaScript 2023 report, Vite leads with 73.3% usage, while Next.js remains widely adopted, with 57.1% of developers having used it.
According to https://prismic.io/blog/vite-vs-nextjs , accessed 12-Aug-2025.

Other strong alternatives include the ASP.NET Core React template and Next.js. The former is simple and pragmatic, but it has an integrated, single codebase. Next.js is also a popular and highly-regarded framework, but its built-in server-side features can introduce unnecessary complexity when a separate backend (like our .NET Core API) is already in use. This would require you to manage two separate back-end stacks.

The core React code of these frameworks is highly reusable regardless of which one you choose. The differences lie in the project structure, build process, and framework-specific features.

You are free to choose any framework you would like. If you use frameworks other than Vite, you need to explore a bit by yourself.

More information can be found below.

ASP.NET Core React Template

- **Primary Purpose:** Integrated Full-Stack web app development.
- **Pros:**
  - Seamless integration of backend and frontend.
  - Single project to manage.
  - Simple development and deployment.
- **Cons:**
  - Less flexible for separate teams.
  - Project is tied to the .NET ecosystem.
- **Popularity:** High in the .NET ecosystem; a standard for projects in enterprise environments using Microsoft tools.
- **Complexity:** Low. The template handles all the integration, making it the easiest to get started with for this specific stack.

- **Scalability:** Scalable, but the frontend and backend are tightly coupled in a single codebase.

Vite + .NET Core

- **Primary Purpose:** Modern Frontend web app development.
- **Pros:**
  - Blazing fast dev server and HMR (Hot Module Replacement).
  - Simple, modern configuration.
  - Best for learning a decoupled architecture.
- **Cons:**
  - Requires managing two separate projects.
  - Need to configure a proxy for API calls.
- **Popularity:** Extremely high. The new standard for starting React frontend projects, quickly replacing Create React App.
- **Professionalism:** Highly professional. This decoupled pattern is a cornerstone of modern web architecture and scales to any project size.
- **Complexity:** Medium. Requires managing two projects and configuring communication (e.g., a proxy).
- **Scalability:** Highly scalable. The decoupled architecture allows you to scale the frontend and backend independently to meet different demands.

Next.js

- **Primary Purpose:** Full-Stack React web framework.
- **Pros:**
  - Industry standard for production-level applications.
  - Built-in features like SSR (Server-Side Rendering), SSG (Static Site Generation), and routing.
  - Excellent performance and SEO.
- **Cons:**
  - High complexity when used with a separate .NET backend.
  - Steep learning curve for beginners.
- **Popularity:** Dominant. A most popular and recommended full-stack React framework for new projects in the industry.
- **Complexity:** High. Introduces server-side concepts, API routes, and a file-based routing system, adding significant overhead for a basic course.
- **Scalability:** Highly scalable. Designed specifically for large, performance-driven projects that require independent rendering strategies.

React Native

- **Primary Purpose:** Cross-Platform Mobile app development.
- **Pros:**
  - Shared codebase for iOS and Android.
  - Uses React knowledge for mobile.
- **Cons:**
  - Not a web development tool.
  - Fundamentally different architecture, making it complex to also use for web.
- **Popularity:** High in mobile development. The leading cross-platform solution for building native mobile apps with React.
- **Complexity:** High. It has a fundamentally different UI architecture from web development.
- **Scalability:** Highly scalable, but specifically for mobile apps. The web side of React Native is not designed for large-scale, standalone web applications.

Create React App (CRA)

- **Primary Purpose:** Officially deprecated Frontend build tool.
- **Pros:**
  - Historically simple setup for beginners.
  - Large library of tutorials and resources.
- **Cons:**
  - CRA was **officially deprecated** by the React team. The official React documentation explicitly advises against using CRA for new projects and recommends tools like Vite or Next.js instead.
  - Slower dev server and build times.
  - No longer actively maintained by the React team.
- **Popularity:** Declining. While it has a legacy user base, its use is rapidly decreasing in favour of Vite.
- **Complexity:** Low. The setup is simple, but its underlying technology is old and cumbersome.
- **Scalability:** Poorly scalable. The slow build and HMR times become significant bottlenecks as projects grow.

Creating the React helloworld project

Verify you node and npm versions.

```
baifanz@Baifan-Mac 6-React-Intro % node -v
v22.18.0
baifanz@Baifan-Mac 6-React-Intro % npm -v
10.2.5
```

If you haven't installed them, or they are not up-to-date, refer to the Demo-Installation-of-software to install/update them.

Go to a folder where you would like to put your React helloworld project. Run the command to create a new project with Vite:

```
npm create vite@latest
```

You may be asked for installing some packages. Press enter or type Y and press enter to confirm and continue.

Then the terminal will ask for the Project name, input the following name and press enter.

```
ReactHelloworld
```

Then the terminal will ask for Package name. We can simply use the default package name (must be all lower case), press tab to use the default name and press enter

Then it will ask for selecting a framework, use arrow key to select React and press enter

```
baifanz@Baifan-Mac 6-React-Intro % npm create vite@latest

◇  Project name:
   ReactHelloworld

◇  Package name:
   reacthelloworld

◆  Select a framework:
   ○ Vanilla
   ○ Vue
   ● React
   ○ Preact
   ○ Lit
   ○ Svelte
   ○ Solid
   ○ Qwik
   ○ Angular
   ○ Marko
   ○ Others
```

Then the terminal will ask for selecting a variant. For now we choose JavaScript to start with learning. Use arrow key to select and press enter.

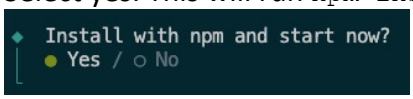
```
◇  Select a framework:
   React

◆  Select a variant:
   ○ TypeScript
   ○ TypeScript + SWC
   ● JavaScript
   ○ JavaScript + SWC
   ○ React Router v7 ↗
   ○ TanStack Router ↗
   ○ RedwoodSDK ↗
   ○ RSC ↗
```

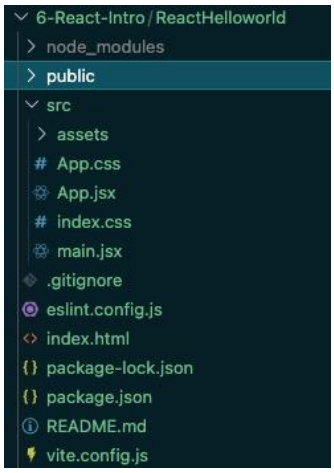
Select ESLint for broader support and industrial standard.

```
◆  Which linter to use?
   ● Oxlint
   ○ ESLint
   ↑/↓ to navigate • Enter: confirm
```

Select yes. This will run `npm install` and `npm run dev` automatically after it.



Wait a bit, and then you will see a lot of files are generated in the project folder ReactHelloworld.



If you selected No, you can do the following
cd to the project:

```
cd ReactHelloworld
```

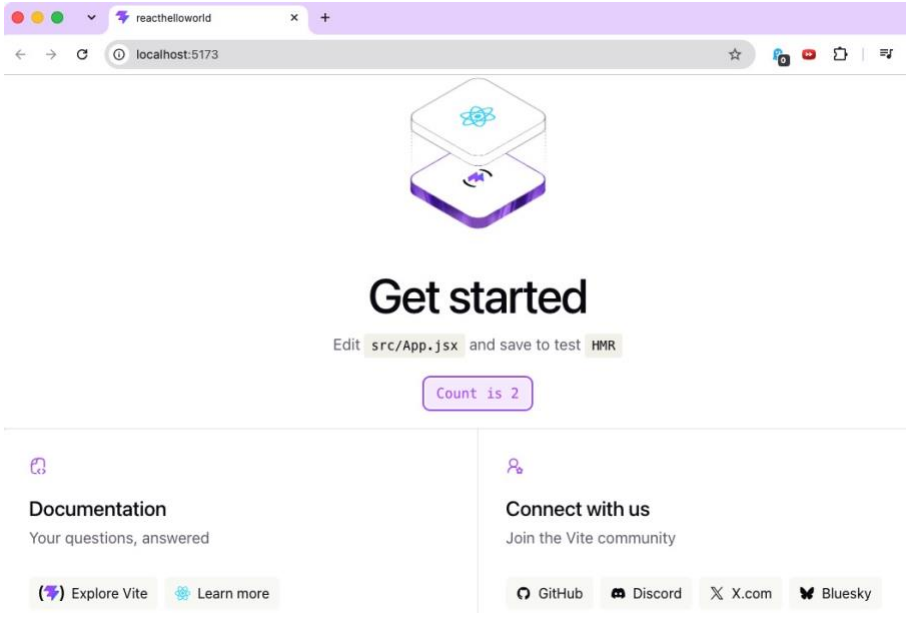
Install node packages by

```
npm install
```

Wait for the package installation, and then run

```
npm run dev
```

then command/ctrl + the shown URL to open the browser, and you will see:



You don’t need to stop running the project. It will auto-recompile. Replace one line code in src/App.jsx as the following code:

App.jsx U

6-React-Intro > ReactHelloworld > src > App.jsx > App

```jsx
import { useState } from 'react'
import heroImg from './assets/hero.png'
import reactLogo from './assets/react.svg'
import viteLogo from './assets/vite.svg'
import './App.css'

function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <section id="center">
        <div className="hero">
          <img src={heroImg} className="base" width="170" height="179" alt="" />
          <img src={reactLogo} className="framework" alt="React logo" />
          <img src={viteLogo} className="vite" alt="Vite logo" />
        </div>
        <div>
          <h1>Hello World</h1>
          <p>
            Edit <code>src/App.jsx</code> and save to test <code>HMR</code>
          </p>
        </div>
        <button
          type="button"
          className="counter"
          onClick={() => setCount((count) => count + 1)}
        >
          Count is {count}
        </button>
      </section>

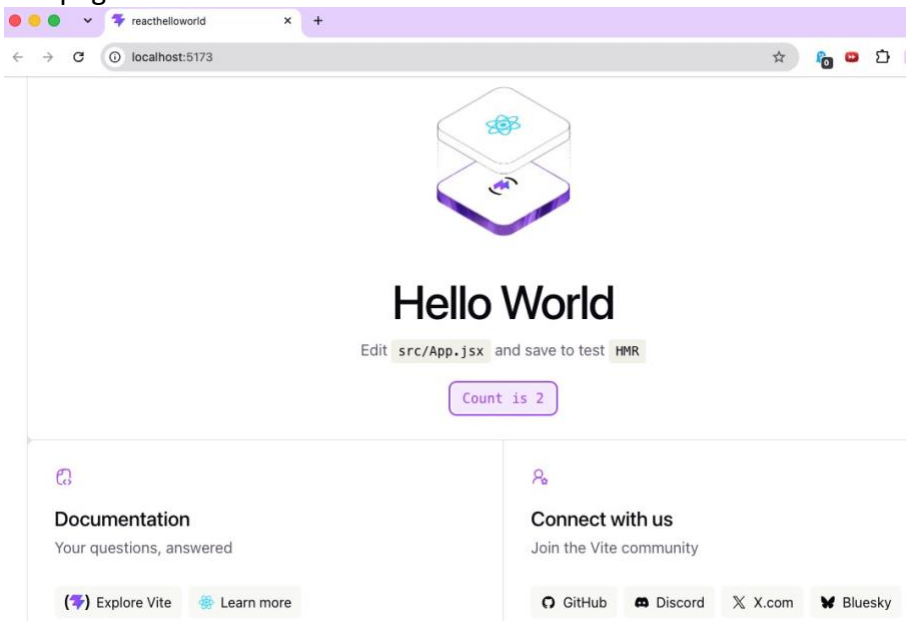
      <div className="ticks"></div>

      <section id="next-steps">
        <div id="docs">…
        </div>
        <div id="social">…
        </div>
      </section>

      <div className="ticks"></div>
      <section id="spacer"></section>
    </>
  )
}

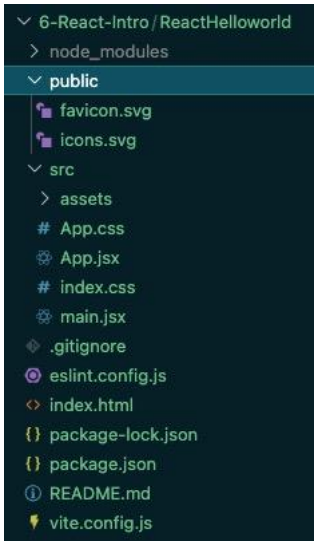
export default App
```

The page now shows:



Explanation of files

We have these files in the helloworld project



node_modules/
This folder contains all the external libraries and packages your project depends on. Think of it as your project's toolbox. It's managed automatically by npm, and you should make changes to the files inside it.

public/
This folder is for static assets that are served directly to the browser. Any files you place here, like images, fonts, or a vite.svg, won't be processed by Vite.

- **public/favicon.svg, icons.sv**: These are **static asset**. Files in the public folder are copied directly to the final build without any processing.

./src This is the most important folder. All of your application's source code, including your React components and styling, will live here.

- **main.jsx**: This is the **entry point** of your application. It's the first file that runs and is responsible for rendering the main App component into the web page.
- **App.jsx**: This is the main **React component** that represents the root of your application's user interface. All other components you create will eventually be rendered inside this one.
- **index.css & App.css**: These are the **styling files**. index.css is for global styles, and App.css is for the specific styles of your main App component.

./src/assets This folder is for other assets like images, fonts, or icons that you want Vite to process during the build.

eslint.config.js: This is the configuration file for **ESLint**, a tool that checks your code for errors and enforces a consistent code style. It helps you write cleaner, more professional code. This file is important to know about, but you don't need to understand every detail right away. For now, you can think of it as a **"code spell checker"**.

index.html: This is the **main web page** of your application. It's the file the browser first loads, and it contains the single HTML element where your entire React application will be injected and rendered.

package-lock.json: This file automatically records the **exact versions** of every package your project uses. It ensures that anyone who installs your project's dependencies will get the exact same versions, preventing inconsistencies and bugs.

package.json: This is the manifest file for your project. It contains key metadata such as your project's name, version, and the scripts you can run. It also defines all external libraries your project depends on.

- **Project Identity:** Contains basic project metadata like name, version, type, and whether the project is private.
- **The "scripts" Section:** Defines custom shortcut commands. When you run npm run dev, npm run build, or npm run lint, npm executes the underlying CLI command (like vite or eslint .) mapping to that key.
- **dependencies:** External packages required for the app to run in production (e.g., react and react-dom). When you run npm install, npm installs these libraries.
- **devDependencies:** Packages needed only during development and building (e.g., build tools like vite and code linters like eslint). These are not needed once the app is deployed.

README.md: This is the **project documentation file**. It is usually the first file a developer sees when they open a project and contains essential information like what the project does, how to install it, and how to run it.

vite.config.js: This is Vite's main **configuration file**. You can customise how your project is built, how the development server runs, and add extra plugins here.